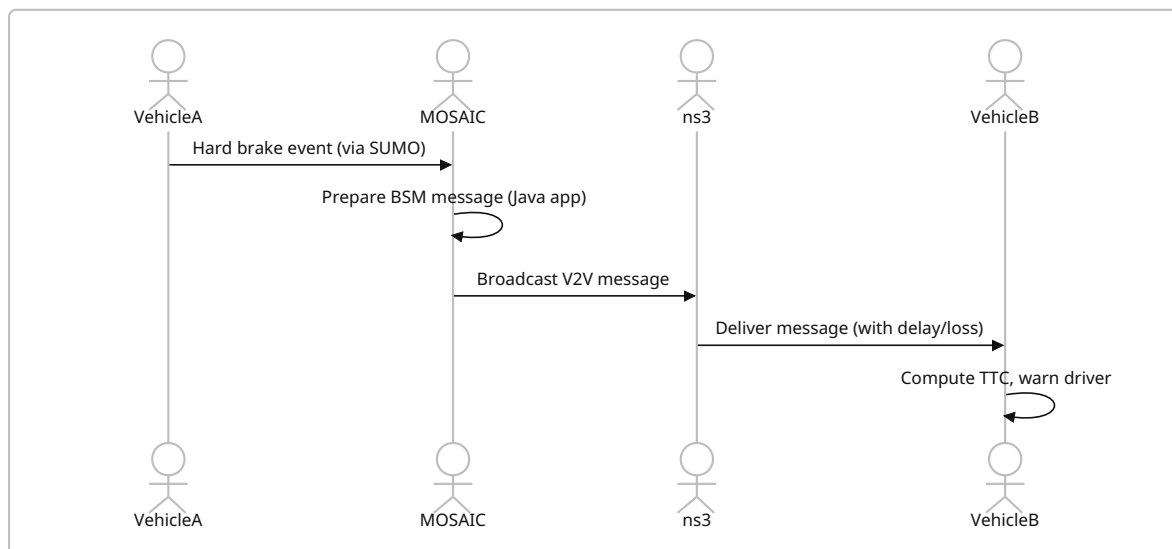


V2V Safety Simulation System – Executive Summary

We propose a **co-simulation framework** for vehicle-to-vehicle (V2V) safety at SIH level, combining traffic, network and device simulators. The **technical stack** uses SUMO (v1.27.1) for road traffic, Eclipse MOSAIC (25.2) to orchestrate federated simulators, ns-3 for wireless communication, and Wokwi's ESP32 simulator as a virtual in-vehicle unit. Java 21 (LTS) is recommended for MOSAIC's Java applications [4†L143-L150]. We will use standard development tools: VS Code (code.visualstudio.com), Python 3 (python.org) with Jupyter for data analysis, and Git (git-scm.com) for version control. Official downloads and versions are cited: SUMO 1.27.1 (Eclipse SUMO) [2†L207-L214], MOSAIC 25.2 (Eclipse MOSAIC) [4†L100-L104], and Wokwi's ESP32 simulator (wokwi.com) [6†L5-L12].

The **system architecture** ties these components: SUMO simulates vehicle dynamics and feeds positions/velocities via the MOSAIC "Sumo Ambassador" (TraCI interface) [22†L136-L144] [22†L147-L155]. MOSAIC's scheduler advances the simulation in lockstep and dispatches the vehicles' states to other federates. A network federate (ns-3) simulates wireless V2V links; MOSAIC "ns-3 Federate" uses XML configs (`ns3_config.json`, `ns3_federate_config.xml`) to set PHY/MAC models [47†L268-L275] [47†L284-L293]. Within MOSAIC each vehicle runs a Java V2X application (EEBL-like collision warning) packaged as a JAR in the `application/` folder [45†L123-L132]. Optionally, the vehicle logic can be prototyped on an ESP32 (simulated in Wokwi); messages are then exchanged via MOSAIC's message bus. A mermaid sequence diagram below illustrates a simple warning flow:



Components and Data Flows: SUMO's output (via `VehicleUpdates`) drives traffic state; MOSAIC logs all interactions (e.g. `VEHICLE_UPDATES`, `V2X_MESSAGE_TRANSMISSION`) to CSV [49†L98-L102]. The MOSAIC `scenario_config.json` ties the simulators together (enabling SUMO and ns-3) [39†L201-L204]. Vehicle applications use MOSAIC's `AdHocModule/CellModule` APIs to send/receive V2X messages. Data flows are: **SUMO↔MOSAIC↔ns-3** (for mobility and comms) and **MOSAIC↔(Java apps)** for decision logic (optionally MOSAIC↔Wokwi as a demo path). We assume urban road networks (SUMO

net) and IEEE 802.11p/C-V2X 5.9 GHz channels in ns-3. Timing is discrete-event: MOSAIC advances SUMO in fixed steps (e.g. 0.1s) [22†L136-L144] , synchronizing network transmissions per ns-3's schedule.

Metrics and Validation: We will measure safety and performance using metrics from the literature: collisions (count), Time-to-Collision (TTC) between vehicles (TTC is the predicted time to impact under constant speed [34†L19-L24]), Packet Delivery Ratio (PDR), end-to-end latency, and driver reaction distance. For example, TTC is a widely-used safety indicator [34†L19-L24] . Failure modes include packet loss (simulated via ns-3 fading/delay models), communication delay, and false positives/negatives in sensor data. We plan a **scaling study** (simulations with ~10, 50, 100, 250, 500 vehicles) to stress-test the system and collect statistics. Statistical analysis (e.g. confidence intervals, significance tests) will ensure robust results. A sample trade-off table (based on DSRC vs C-V2X) might compare latency, range, packet loss under load:

Technology	Range	Latency	Packet Loss (high load)
DSRC (802.11p)	~50–500 m	~50–100 ms [53†L345-L348]	Moderate
C-V2X (PC5)	~300–1000 m	~1–10 ms	Low (scheduled MAC)
5G NR-V2X/5G	>1 km	<5 ms	Very low

Implementation Plan (milestones): We will script the build and run steps. (1) **Install tools:** Java 21 (Eclipse Temurin) [4†L143-L150] , SUMO 1.27.1 [2†L207-L214] , MOSAIC 25.2 [4†L100-L104] , ns-3 (v3.36.x), and Wokwi/Arduino. (2) **Prepare scenario:** use `scenario-convert.sh` to load the SUMO net/route into a SQLite DB [43†L39-L46] . Create a MOSAIC scenario folder with subfolders (`application/` , `mapping/` , `sumo/` , plus `scenario_config.json`) [43†L153-L161] [45†L123-L132] . (3) **Edit configs:** in `sumo/` , place `<name>.net.xml` , `.rou.xml` , `.sumocfg` ; write `sumo_config.json` to point MOSAIC to SUMO's config. In `mapping/` , define `mapping_config.json` if needed. In root, edit `scenario_config.json` to enable SUMO and ns-3 federates [39†L201-L204] , set simulation duration, etc. In `application/` , place `application_config.json` (defining apps) and the Java JAR(s) of our safety app [45†L123-L132] . In `ns3/` , place `ns3_config.json` (ambassador filters) and `ns3_federate_config.xml` (802.11p stack settings) [47†L268-L275] . (4) **Run simulation:** use MOSAIC's `mosaic.sh` or `mosaic.bat` to start; it will spawn SUMO and the ns-3 federate. A sample run script (`run_v2v.bat`) will automate building and launching. We will validate using MOSAIC's **FileOutput** (CSV logs of interactions [49†L98-L102]) and **StatisticsOutput** (CSV summaries) [50†L124-L132] .

Validation and Analysis: Output logs (CSV) from MOSAIC (e.g. `VehicleUpdates.csv` , `V2X_Message.csv` , `Stats*.csv`) will be parsed in Python. For example, using Pandas/Matplotlib one can compute TTC per vehicle or PDR:

```
import pandas as pd
df = pd.read_csv('output/VehicleUpdates.csv', sep=';')
# Compute TTC from positions and speeds (pseudo-code)
df['TTC'] = compute_ttc(df['speed'], df['distance'])
print(df[['time', 'vehicle_id', 'TTC']].head())
```

Plotting libraries (matplotlib, Seaborn or Plotly) will visualise time-series (e.g. collision distances, PDR vs time). For dashboards, a Python-based stack is suitable: e.g. Plotly Dash or Streamlit for interactive plots, or Grafana (with InfluxDB) for live metrics. The data schema is simple (timestamped events), so

formats like CSV or InfluxDB line protocol can be used. We will log events like

`V2X_MESSAGE_TRANSMISSION` and `VEHICLE_UPDATES` [49†L104-L113] to analyse message rates and safety margins.

Risks, Resources, Hardware: Simulation may be CPU-heavy (especially ns-3). We recommend a multi-core machine (≥ 8 cores, 16–32 GB RAM) or cloud VM. SSDs for fast I/O. For optional hardware, SDR boards (e.g. USRP N210 or HackRF) could be used to emit real IEEE 802.11p or C-V2X signals in a later phase, and Commercial Off-the-Shelf OBU/RSU units could validate real-world comms. However, these are advanced extensions; initial focus is fully virtual. The team will need Java skills (for MOSAIC apps), plus Python for analysis. Key risks include synchronization bugs (MOSAIC timing) and message handling logic. We will mitigate by starting with small scenarios (e.g. “Highway” or “Barnim” examples), then scaling up.

Reading List (Key Sources):

- Eclipse MOSAIC Official Docs (config files, tutorials) [43†L39-L46] [47†L268-L275] .
- SUMO 1.27.1 Download and docs [2†L207-L214] .
- Wokwi ESP32 Simulator (web resource) [6†L5-L12] .
- NIST/IEEE on V2V safety (EEBL application) [64†L59-L67] .
- Alupoaei & Caruntu (2025), “*Optimizing Urban Traffic Efficiency and Safety via V2X*”, MOSAIC+SUMO study [28†L34-L42] [28†L44-L53] .
- Streetscope on Time-to-Collision [34†L19-L24] .
- Patents/papers on DSRC vs C-V2X performance [53†L345-L348] .
- SAE J2735 (BSM standard) and ETSI CAM specifications for message formats.

These references and the architecture/plan above provide an end-to-end roadmap. In summary, we will assemble and configure the software stack, design scenarios (e.g. “sudden brake in convoy”), implement the V2X app logic, run co-simulations at scale, and validate with statistical analysis of the MOSAIC logs. This approach is grounded in official sources and recent research [28†L34-L42] [64†L59-L67] to ensure a robust SIH-ready V2V safety demo.
